

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Архитектура вычислительных систем

К защите допустить:

И.О. Заведующего кафедрой
информатики

_____ С. И. Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту

на тему

**ЭФФЕКТИВНАЯ РЕАЛИЗАЦИЯ АЛГОРИТМОВ НА ПРОЦЕССОРАХ
НЕСКОЛЬКИХ АРХИТЕКТУР С УЧЕТОМ ИХ ОСОБЕННОСТЕЙ**

БГУИР КП 1-40 04 01 009 ПЗ

Студент

Е. С. Кахновский

Руководитель

А. А. Калиновская

Нормоконтролер

А. А. Калиновская

Минск 2023

СОДЕРЖАНИЕ

Введение	5
1 Архитектуры вычислительных систем	6
1.1 Архитектура вычислительной системы <i>x86</i>	6
1.2 Архитектура вычислительной системы <i>ARM</i>	13
2 Платформа программного обеспечения.....	20
2.1 <i>VirtualBox (Oracle VM VirtualBox)</i>	20
3 Теоретическое обоснование разработки программного продукта.....	24
4 Проектирование функциональных возможностей программы	28
4.1 Выбор подходящих алгоритмов	28
4.2 Расстояние Левенштейна	29
4.3 Быстрое преобразование Фурье	35
5 Сравнение эффективности алгоритмов	40
5.1 Основные сведения	40
5.2 Анализ эффективности алгоритма Левенштейна	40
5.3 Анализ эффективности алгоритма БПФ.....	42
Заключение	45
Список литературных источников	46
Приложение А (обязательное) Исходный код программы	48
Приложение Б (обязательное) Функциональная схема алгоритма.....	52
Приложение В (обязательное) Блок-схема алгоритма	53
Приложение Г (обязательное) Скриншоты сравнений производительности ..	54
Приложение Д (обязательное) Ведомость документов.....	55

ВВЕДЕНИЕ

Современные вычислительные системы характеризуются многообразием архитектур, которые различаются по своим техническим характеристикам и функциональным особенностям. Среди ключевых компонентов вычислительных систем, архитектура процессоров играет важную роль, определяя их способность обрабатывать данные и выполнять разнообразные вычислительные задачи. В этом контексте, эффективная реализация алгоритмов на различных архитектурах процессоров становится неотъемлемой частью развития информационных технологий и приложений, а также отражает важное направление исследований в современной компьютерной науке и инженерии. Данный курсовой проект нацелен на глубокое исследование этой проблематики с учетом индивидуальных особенностей различных архитектур процессоров.

С постоянным увеличением вычислительных мощностей современных компьютеров и расширением областей применения вычислений, оптимизация алгоритмов для различных архитектур становится неотъемлемой частью разработки программного обеспечения. Важно учесть, что архитектурные особенности, такие как архитектуры *CISC* (*Complex Instruction Set Computer*) и *RISC* (*Reduced Instruction Set Computer*) требуют специфических подходов для достижения максимальной производительности. Таким образом, актуальность данной работы заключается в необходимости оптимизации алгоритмов с учетом конкретных архитектур процессоров.

Целью данного курсового проекта является исследование и разработка эффективных реализаций алгоритмов на различных архитектурах: *x86*, *ARM*.

Для достижения данной цели поставлены следующие задачи:

- 1 Провести детальный анализ архитектур *x86* и *ARM*.
- 2 Разработать разнообразные алгоритмы, которые подходят для выполнения на каждой из выбранных архитектур. Учесть специфику архитектур для оптимизации алгоритмов.
- 3 Реализовать разработанные алгоритмы на каждой архитектуре и провести их тестирование с использованием соответствующих сред разработки и инструментов для каждой платформы.
- 4 Провести сравнительный анализ производительности алгоритмов на разных архитектурах и сделать выводы о преимуществах и недостатках каждой из них.

Пояснительная записка оформлена в соответствии с СТП 01-2017 [1].

1 АРХИТЕКТУРЫ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ

1.1 Архитектура вычислительной системы x86

1.1.1 Структура и архитектура вычислительной системы x86

Структура и архитектура вычислительной системы x86 представляют собой сложную систему, включающую центральный процессор (*CPU*), системную память, системную шину и периферийные устройства [2].

Рассмотрим более подробно каждый из этих компонентов:

1 Центральный процессор (*CPU*) является главным вычислительным устройством в системе с архитектурой x86. *CPU* представляет собой микропроцессор, который спроектирован для выполнения инструкций и управления выполнением программ.

В его состав входят несколько ключевых компонентов:

1.1 *ALU* (Арифметико-логическое устройство) выполняет арифметические и логические операции, такие как сложение, вычитание, умножение, деление и сравнение.

1.2 Регистры — это небольшие хранилища данных, находящиеся непосредственно внутри процессора. Они используются для хранения промежуточных результатов и данных, с которыми процессор работает на данный момент.

1.3 Управляющая логика *CPU* управляет выполнением инструкций, выбором следующей инструкции для выполнения и управлением доступом к памяти и периферийным устройствам.

2 Системная память (*RAM*) используется для хранения данных и программ, которые *CPU* может читать и записывать. Системная память является важной частью вычислительной системы и обеспечивает временное хранение данных, необходимых для выполнения программ.

3 Системная шина (также известная как системная магистраль) соединяет различные компоненты системы, включая *CPU*, системную память и периферийные устройства. Она служит для передачи данных и команд между этими компонентами и обеспечивает их взаимодействие.

4 Периферийные устройства включают в себя все устройства, которые не являются частью *CPU*, памяти или системной шины. Это могут быть клавиатуры, мыши, мониторы, принтеры, жесткие диски, сетевые адаптеры и многое другое. *CPU* и периферийные устройства взаимодействуют через системную шину.

Архитектура *x86* также имеет различные режимы работы, такие как реальный режим, защищенный режим и длинный режим (для 64-битных версий). Режимы определяют, как *CPU* обрабатывает инструкции и как она имеет доступ к памяти.

Архитектура *x86*, в том числе и её расширение *x86_64*, использует комплексный набор команд (*CISC*), что означает, что она предоставляет широкий и разнообразный набор инструкций для выполнения различных операций.

CISC, или комплексный набор команд (*Complex Instruction Set Computing*) (рисунок 1.1), представляет собой архитектурный подход, при котором процессор предоставляет обширный и разнообразный набор инструкций для выполнения различных задач. Одной из главных особенностей *CISC* является то, что инструкции могут выполнять сложные операции, включая арифметические и логические действия, операции с памятью, управление периферийными устройствами и многое другое, что позволяет реализовывать высокоуровневые операции с минимальным количеством команд.

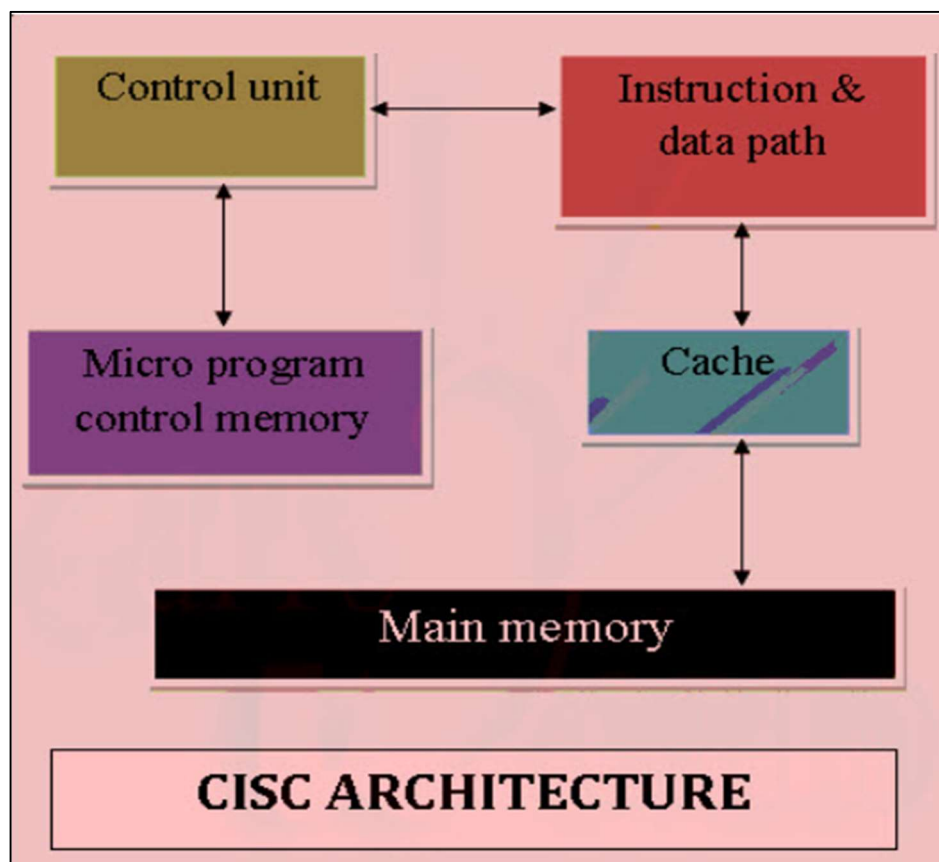


Рисунок 1.1 – Архитектура *CISC*

Преимущества *CISC* включают в себя более высокий уровень абстракции, что делает программирование более удобным для разработчиков. Сложные инструкции *CISC* позволяют выполнять множество операций с минимальным объемом кода, что особенно важно при написании сложных программ. Кроме того, *CISC*-процессоры могут быть эффективны в использовании памяти, так как одна инструкция может выполнять множество операций над данными, сэкономя тактовые циклы [4].

Однако у *CISC* есть и недостатки. Сложность инструкций может привести к более длительным циклам выполнения, что снижает скорость работы процессора. Кроме того, сложные инструкции могут затруднить проектирование процессора и увеличить его сложность. Это также может сказаться на энергоэффективности процессора.

Формула оценки производительности процессора:

$$\frac{time}{program} = \frac{time}{cycle} * \frac{cycles}{instruction} * \frac{instructions}{program}, \quad (1)$$

где $\frac{time}{program}$ — это общее время выполнения программы. Это сумма времени, затраченного на выполнение всех инструкций в программе.

$\frac{time}{cycle}$ — это время, затраченное на выполнение одного цикла(такта) процессора. Это показатель, который указывает, сколько времени требуется процессору для выполнения одной базовой операции.

$\frac{cycles}{instruction}$ — это среднее количество циклов процессора, требуемых для выполнения одной инструкции. Это отражает эффективность выполнения инструкций процессором.

$\frac{instructions}{program}$ — это общее количество инструкций в программе. Это отражает сложность программы и количество операций, которые процессор должен выполнить.

CISC-подход направлен на минимизацию количества инструкций в программе. Жертвуя при этом количеством циклов на одну инструкцию.

Общая структура и архитектура вычислительной системы *x86* обеспечивают ей высокую универсальность и способность обрабатывать разнообразные задачи, что сделало ее одной из наиболее популярных архитектур для персональных компьютеров и серверов.

1.1.2 История архитектуры x86

Архитектура *x86*, также известная как *Extended 86*, имеет долгую и интересную историю [3]. Она связана с компанией *Intel*, одним из мировых лидеров в производстве процессоров и полупроводников.

Основные этапы истории архитектуры *x86*:

1 *Intel 8086* (1978 год): Зарождение архитектуры *x86* началось с выпуска процессора *Intel 8086*. Этот процессор был 16-битным и мог адресовать до 1 МБ оперативной памяти. Он стал отправной точкой для развития архитектуры *x86*.

2 *Intel 80286* (1982 год): В 1982 году был представлен процессор *Intel 80286*, который увеличил объем адресуемой оперативной памяти до 16 МБ и принес с собой существенные улучшения в производительности и функциональности.

3 *Intel 80386* (1985 год): В 1985 году *Intel* выпустила процессор *Intel 80386*, который внёс революционные изменения. Этот процессор стал первым 32-битным процессором семейства *x86* и мог адресовать более 4 ГБ оперативной памяти. Он также внёс поддержку защищенного режима, что было важным шагом для поддержки многозадачных операционных систем.

4 *Intel 486* (1989 год): Процессоры семейства *Intel 486* продолжили улучшать производительность и внесли оптимизации, которые сделали выполнение инструкций более быстрым.

5 Процессоры *Pentium* (1993 год): С началом 1990-х годов процессоры *Pentium* от *Intel* стали символом высокой производительности и получили широкое признание на рынке. Они представили множество новых возможностей и открыли дорогу к высокоскоростным вычислениям.

6 64-битные процессоры (2000-е годы): В более поздние годы архитектура *x86* развивалась в сторону 64-битных процессоров, таких как *Intel Itanium* и *AMD64*. Эти процессоры смогли обрабатывать большие объемы оперативной памяти и обеспечивать более высокую производительность.

Архитектура *x86* стала широко используемой не только в настольных компьютерах, но и в серверах и мобильных устройствах. Сегодня 64-битные процессоры семейства *x86_64* (или *x64*) доминируют на рынке и предоставляют мощные вычислительные возможности.

1.1.3 Версии и достоинства архитектуры x86

Архитектура x86 имеет несколько версий, включая 32-битную (x86) и 64-битную (x86_64 или x64) версии. Каждая из них имеет свои особенности и преимущества:

1 32-битная архитектура (x86): Ограничение по объему памяти: 32-битная архитектура может адресовать максимум 4 ГБ оперативной памяти. Это ограничение может быть преодолено с использованием различных техник, таких как *Physical Address Extension (PAE)*, но базовая архитектура ограничивает доступный объем памяти. Обратная совместимость: 32-битные процессоры x86 сохраняют обратную совместимость, что означает, что они могут выполнять старые 16-битные и 32-битные программы, что важно для поддержки старого программного обеспечения.

2 64-битная архитектура (x86_64 или x64): Увеличенный объем памяти: 64-битная архитектура позволяет адресовать гораздо больший объем оперативной памяти, практически без ограничений (до 18,4 миллионов терабайт). Улучшенная производительность: 64-битные процессоры способны обрабатывать большие объемы данных более эффективно, что увеличивает производительность при выполнении сложных задач и вычислений. Поддержка новых технологий: 64-битная архитектура обеспечивает поддержку новых технологий, таких как более продвинутые алгоритмы шифрования и расширенные возможности виртуализации.

Достоинства архитектуры x86:

1 Универсальность: x86-совместимые процессоры широко поддерживаются операционными системами и приложениями, что делает их универсальными и совместимыми с большим количеством программного обеспечения.

2 Мощность: Многие процессоры x86 обладают высокой производительностью и способностью выполнять сложные вычисления, что делает их подходящими для широкого спектра задач.

3 Обратная совместимость: Процессоры x86 сохраняют обратную совместимость с более старыми программами, что важно для бизнеса и конечных пользователей, которые используют старое программное обеспечение.

Выбор архитектуры x86 в зависимости от конкретных задач может зависеть от требований к объему памяти, производительности и совместимости с программами.

1.1.4 Обоснование выбора вычислительной системы x86

Обоснование выбора архитектуры x86 для исследования в данной курсовой работе также подкрепляется использованием и тестированием процессора *AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz* (рисунок 1.2) в качестве представителя данной архитектуры [5].

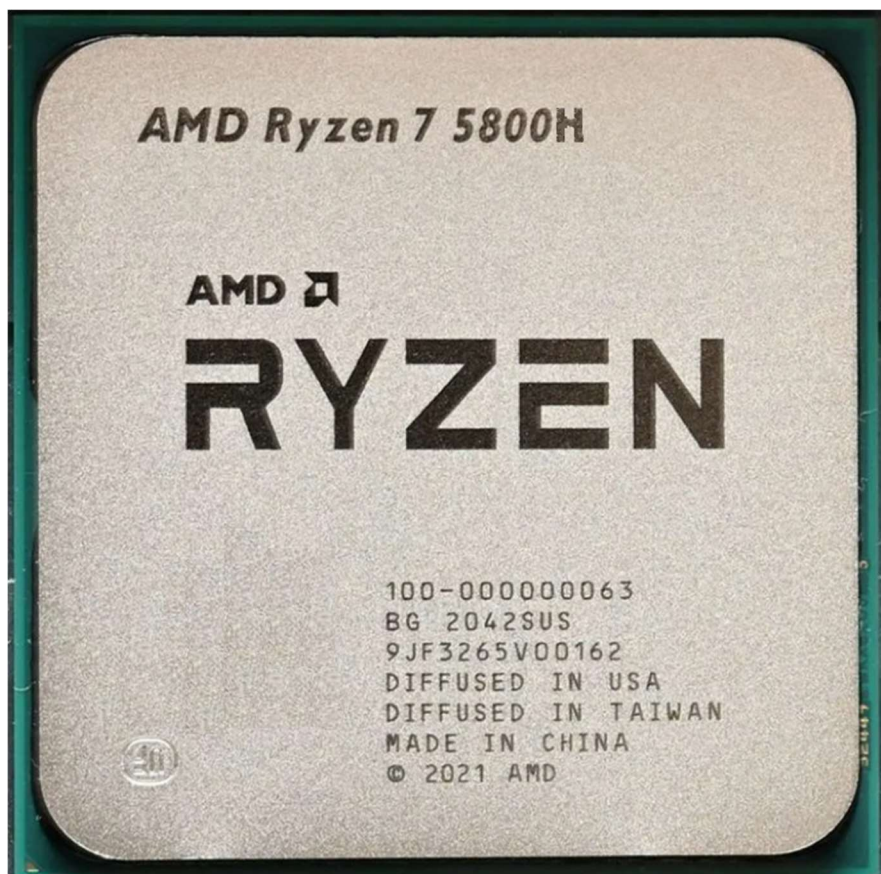


Рисунок 1.2 – Процессор *AMD Ryzen 7 5800H*

AMD Ryzen 7 5800H – мобильный процессор, разработанный в 2021 году для интеграции в ноутбуки, и предлагает выдающуюся производительность и разносторонние возможности. Вот более подробное описание его характеристик:

1 Модель: *AMD Ryzen 7 5800H*.

2 Базовая частота: *3.20 GHz*.

3 Количество ядер: 8.

4 Количество потоков: 16.

5 Технология *Simultaneous Multithreading (SMT)* активирует поддержку выполнения двух потоков на одном физическом ядре, повышая общую производительность.

6 Технология процесса: 7 нм – это передовая технология, обеспечивающая энергоэффективность и высокую производительность.

7 Кэш L3: 16 МБ – обеспечивает быстрый доступ к данным, повышая эффективность обработки информации.

8 Сокет: *FP6 (AM4)* – это разъем, который обеспечивает совместимость с платформой *AM4*, что может быть важным для возможности апгрейда.

9 Тепловыделение (*TDP*) – обычно 45 Вт, но фактическое тепловыделение может варьироваться в зависимости от конкретной конфигурации ноутбука. Это важный параметр для управления температурой и энергопотреблением.

10 Модель интегрированной графики: *Radeon Graphics*

11 Поддержка виртуализации: *AMD-V* – обеспечивает поддержку виртуализации, что важно для запуска виртуальных машин.

12 Поддержка расширений инструкций: Включая *AMD64/EM64T* и другие – это гарантирует совместимость с современными программами и операционными системами.

Выбор *AMD Ryzen 7 5800H* для исследования обусловлен совокупностью факторов, делающих архитектуру *x86* привлекательной для анализа и экспериментов.

1 Широкая распространенность: Процессор *AMD Ryzen 7 5800H*, основанный на архитектуре *x86*, представляет собой часть одного из самых распространенных семейств процессоров в мире. Эта широкая распространенность делает его отличным представителем для анализа, поскольку результаты исследований на этом процессоре могут быть легко обобщены на более широкую аудиторию.

2 Доступность и ресурсы: *AMD Ryzen 7 5800H* предоставляет исследователям доступ к обширным ресурсам, связанным с архитектурой *x86*. Легкая доступность и широкое распространение этого процессора упрощают проведение экспериментов и тестирование различных алгоритмов и приложений.

3 Универсальность: Архитектура *x86*, представленная процессором *AMD Ryzen 7 5800H*, обеспечивает поддержку множества операционных систем и программного обеспечения. Это позволяет исследователям работать в различных средах, что важно для всестороннего анализа производительности и совместимости.

4 Обратная совместимость: Процессор *AMD Ryzen 7 5800H* сохраняет обратную совместимость с более ранними программами, что позволяет использовать старые приложения на современном оборудовании без

необходимости их модернизации.

5 Инструменты разработки и оптимизации: для архитектуры *x86*, включая процессор *AMD Ryzen 7 5800H*, доступны многочисленные инструменты разработки, отладки и оптимизации. Это предоставляет исследователям эффективные средства для изучения производительности и оптимизации программ.

В заключении, выбор архитектуры *x86* для исследования обоснован множеством факторов, среди которых широкая распространенность, доступность ресурсов, универсальность, обратная совместимость и наличие эффективных инструментов разработки. Процессор *AMD Ryzen 7 5800H*, как представитель этой архитектуры, является отличным выбором для демонстрации этих принципов. Его широкое использование, универсальность в поддержке операционных систем и приложений, а также способность к эффективной обработке задач делают его репрезентативным объектом для изучения и анализа. Такой подход позволяет получить более глубокое понимание особенностей архитектуры *x86* и её влияния на разработку и эффективность алгоритмов на примере современного процессора *AMD Ryzen 7 5800H*.

1.2 Архитектура вычислительной системы *ARM*

1.2.1 Структура и архитектура вычислительной системы *ARM*

Архитектура *ARM (Advanced RISC Machine)* представляет собой инновационную и эффективную архитектуру процессоров, разработанную с учетом высокой производительности и низкого энергопотребления. Вот более подробная структура и архитектурные особенности систем, использующих архитектуру *ARM* [6]:

1 Процессоры *ARM*: Центральным компонентом системы на базе архитектуры *ARM* является процессор *ARM*. Он выполняет все вычислительные задачи и управляет выполнением программ. Процессоры *ARM* отличаются высокой эффективностью и энергоэффективностью. Они используют принцип архитектуры *RISC (Reduced Instruction Set Computer)*, что означает, что они имеют набор относительно простых и быстрых инструкций для выполнения операций.

Архитектура *RISC* (рисунок 1.3), включая её расширение, такое как *RISC-V*, применяет принцип уменьшенного набора команд, предоставляя более простой и ограниченный набор инструкций для выполнения операций.

Особенности *RISC* включают в себя простые инструкции, которые обычно выполняют базовые операции, такие как загрузка, сохранение, арифметические и логические действия. У *RISC* процессоров обычно отсутствуют сложные инструкции, которые характерны для *CISC*, что способствует более высокой скорости выполнения инструкций [7].

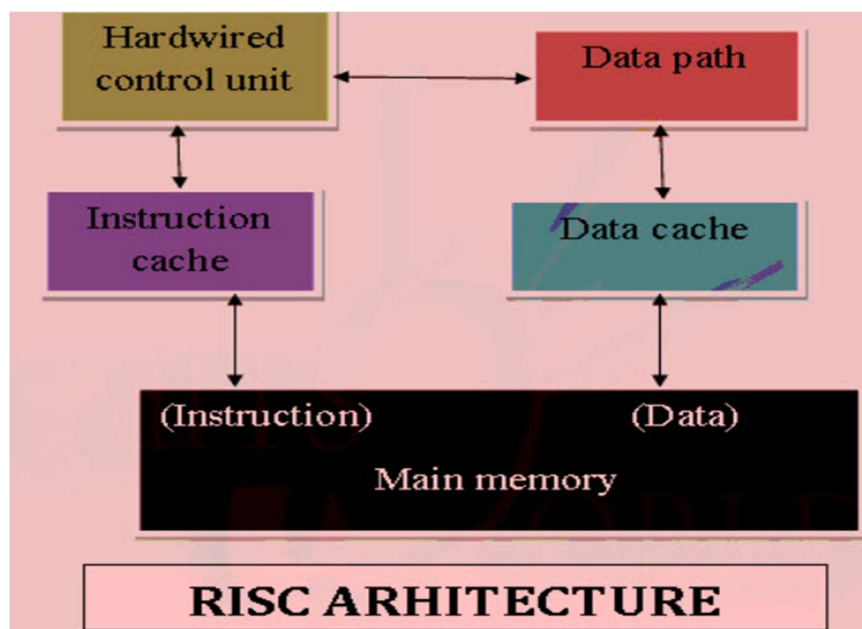


Рисунок 1.3 – Архитектура *RISC*

Преимущества *RISC* включают в себя более короткие циклы выполнения, что приводит к повышенной производительности. Отсутствие сложных инструкций также облегчает проектирование процессора, уменьшая его сложность и обеспечивая более эффективное использование энергии.

Кроме того, *RISC* способствует легкости в оптимизации компиляторов и повышению параллелизма инструкций.

Однако у *RISC* архитектуры также есть свои недостатки. Например, из-за ограниченного набора инструкций, выполнение некоторых сложных операций может потребовать большего количества базовых инструкций. Это может привести к увеличению объема кода и использованию памяти. Кроме того, для реализации некоторых высокоуровневых операций может потребоваться больше инструкций, что влияет на эффективность использования тактовых циклов. В целом, архитектура *RISC* ориентирована на обеспечение простоты и эффективности, что особенно актуально в контексте высокопроизводительных вычислений и энергоэффективных решений.

Исходя из формулы (1) можно сделать вывод, что *RISC*-подход делает все наоборот *CISC*-подходу, уменьшая количество циклов на одну инструкцию в программе.

2 Память: Системы на базе архитектуры *ARM* используют оперативную память (*RAM*) для временного хранения данных и программ. В качестве постоянного хранилища данных могут использоваться флеш-память или карты памяти. Это обеспечивает быстрый доступ к данным и программам.

3 Системная шина: Системная шина является коммуникационным каналом, который соединяет различные компоненты системы. Она обеспечивает передачу данных и команд между процессором *ARM*, памятью и устройствами ввода/вывода. Это позволяет компонентам системы взаимодействовать и совместно работать.

4 Инструкционный набор и архитектура: Архитектура *ARM* имеет четко определенный набор инструкций, что делает ее предсказуемой и эффективной. Это позволяет процессорам *ARM* быстро выполнять операции и обеспечивать высокую производительность.

5 Многообразие вариантов: Семейство процессоров *ARM* включает различные варианты, которые могут быть настроены и адаптированы для разных целей. Например, процессоры *Cortex-A* предназначены для высокой производительности и применяются в смартфонах и планшетах, в то время как *Cortex-M* используются в микроконтроллерах для встроенных систем.

Структура и архитектура систем, основанных на архитектуре *ARM*, продемонстрировали свою эффективность и применимость в современных вычислительных устройствах и являются одними из наиболее популярных и распространенных в мире.

1.2.2 История архитектуры *ARM*

История архитектуры *ARM* (*Advanced RISC Machine*) является важной историей в мире вычислительных технологий, и она началась в 1980-х годах.

Архитектура *ARM* была разработана в Великобритании компанией *Acorn Computers* в 1983 году. Первоначально она создавалась для использования в персональных компьютерах *Acorn Archimedes*. Эти компьютеры были одними из первых, которые использовали архитектуру *ARM*, и они были представлены на рынке в 1987 году.

В 1990 году компания *Advanced RISC Machines Ltd.* (сокращенно *ARM Ltd.*) была основана с целью продвижения архитектуры *ARM* на рынке и лицензирования ее другим компаниям. Этот шаг оказался важным, поскольку он позволил другим производителям создавать свои собственные процессоры

на базе архитектуры *ARM*.

Архитектура *ARM* продолжала развиваться и усовершенствоваться. В 1991 году появилась первая версия *ARM*, известная как *ARMv3*. Следующие версии, такие как *ARMv4* и *ARMv5*, добавили новые инструкции и возможности, что улучшило производительность и функциональность процессоров *ARM*.

ARM Ltd. начала разрабатывать различные семейства процессоров, каждое из которых было настроено на определенные приложения. Например, семейство *Cortex-A* предназначено для высокопроизводительных приложений, *Cortex-R* для систем реального времени, а *Cortex-M* для микроконтроллеров.

Архитектура *ARM* начала завоевывать популярность в мире мобильных устройств, начиная с КПК и карманных компьютеров в 1990-х годах. Ее низкое энергопотребление и высокая производительность сделали ее идеальным выбором для смартфонов и планшетов. Позже, в 2010-х годах, процессоры *ARM* начали проникать и на рынок серверов, что усилило ее популярность.

Одной из ключевых особенностей архитектуры *ARM* является модель лицензирования. Компания *ARM Ltd.* лицензирует свои технологии другим компаниям, что позволяет им создавать собственные процессоры и чипы на базе архитектуры *ARM*. Это привело к широкому распространению процессоров *ARM* и разнообразию вариантов на рынке.

В настоящее время процессоры *ARM* широко применяются в мобильных устройствах, планшетах, смартфонах, автомобилях, встроенных системах, серверах и многих других областях. Семейство *Cortex-A*, например, предоставляет высокую производительность, в то время как *Cortex-M* используется во многих микроконтроллерах.

История архитектуры *ARM* свидетельствует о ее значительном влиянии на мир вычислительной технологии и ее способности адаптироваться для различных потребностей рынка. Ее эффективность и энергоэффективность сделали ее одной из самых востребованных архитектур в современных устройствах.

1.2.3 Версии и достоинства архитектуры *ARM*

Архитектура *ARM* (*Advanced RISC Machine*) имеет разнообразные версии и варианты, которые адаптированы для различных типов вычислительных устройств и задач. Ниже представлены основные версии и достоинства архитектуры *ARM*:

Версии архитектуры *ARM*:

1 Первая версия архитектуры *ARM* была представлена в 1985 году. Она

была разработана для использования в компьютерах *Acorn Archimedes* и была 32-битной.

2 Версия *ARMv2*, выпущенная в конце 1980-х, добавила поддержку кэш-памяти и арифметических инструкций.

3 Версия *ARMv3* внесла улучшения в архитектуру, включая предварительное выполнение инструкций. Эта версия была успешно использована в первых семействах процессоров *ARM*.

4 Версия *ARMv4* добавила 32-битное расширение и более широкий набор инструкций, что увеличило производительность и функциональность.

5 Версия *ARMv5* внесла еще больше улучшений в архитектуру и включила поддержку многозадачности и Java-инструкций.

6 Версия *ARMv6* стала базовой для семейства процессоров *ARM11*, которые широко применялись в смартфонах и мобильных устройствах.

7 Версия *ARMv7* внесла значительные улучшения, включая поддержку 64-битных вычислений (в семействе *Cortex-A*), виртуализацию и многозадачность. Эта версия использовалась в современных смартфонах и планшетах, а также во встроенных системах.

8 Версия *ARMv8* представила архитектуру *ARM64* (или *AArch64*), которая полностью поддерживает 64-битные вычисления. Это открыло двери для использования архитектуры *ARM* в серверных и высокопроизводительных вычислениях.

Достоинства архитектуры *ARM*:

1 Энергоэффективность: Одним из ключевых достоинств архитектуры *ARM* является ее энергоэффективность. Это делает ее идеальным выбором для мобильных устройств, таких как смартфоны и планшеты, где продолжительность работы от аккумулятора критична.

2 Многообразие вариантов: Семейство процессоров *ARM* включает в себя различные варианты для разных целей. *Cortex-A* предназначены для высокой производительности, *Cortex-R* – для систем реального времени, а *Cortex-M* – для микроконтроллеров.

3 Широкое применение: Архитектура *ARM* широко используется во многих областях, включая мобильные устройства, встроенные системы, автомобили, серверы и более. Это обеспечивает совместимость и доступность множества устройств и приложений.

4 Сообщество и поддержка: существует большое сообщество разработчиков и компаний, работающих с архитектурой *ARM*. Это обеспечивает поддержку, разнообразие инструментов и техническую экспертизу.

5 Большой рынок: *ARM* доминирует на рынке мобильных устройств и начинает расширяться на рынок серверов. Это делает архитектуру *ARM* важной с точки зрения выбора и оптимизации вычислительных решений.

Архитектура *ARM* остается фундаментальным и важным компонентом в мире современных вычислительных технологий, постоянно эволюционируя и поддерживая высокий уровень производительности и эффективности в разнообразных областях ее применения. Непрерывные улучшения и инновации в рамках архитектуры *ARM* не только укрепляют ее позицию в сфере мобильных устройств, но также способствуют успешному внедрению в другие области, такие как встроенные системы, интернет вещей (*IoT*), серверы с энергоэффективными решениями, обеспечивая надежность и производительность на высоком уровне.

1.2.4 Обоснование выбора вычислительной системы *ARM*

Обоснование выбора архитектуры *ARM* для исследования в данной курсовой работе также подкрепляется использованием и тестированием процессора *Apple M1* (рисунок 1.4) в качестве представителя данной архитектуры [8].



Рисунок 1.4 – Процессор *Apple M1*

Apple M1 – это процессор, разработанный *Apple* для использования в их компьютерах *Mac*. Полный список характеристик для данного процессора:

- 1 *ARM-based* архитектура.
- 2 техпроцесс – 5 нм.
- 3 8 высокопроизводительных ядер, где 4 ядра "*Performance*" (высокая производительность) и 4 ядра "*Efficiency*" (эффективность для задач с низким

энергопотреблением).

4 Встроенное графическое ядро с 8 ядрами *GPU*.

5 Нейронные движки для обработки искусственного интеллекта и машинного обучения.

6 *Unified Memory Architecture* (Общая архитектура памяти). Объединенная архитектура памяти позволяет разделять память между центральным процессором и графическим процессором, что улучшает общую производительность.

7 Использование технологии *Big.Little* для сочетания высокой производительности и эффективности энергопотребления.

8 Встроенный блок ускорения для обеспечения высокой производительности шифрования данных.

9 Поддержка *Thunderbolt / USB 4*.

10 Полная совместимость с операционной системой *macos Big Sur* и новее.

11 Возможность запуска приложений, оптимизированных для архитектуры *ARM*, а также приложений *Intel x86* через *Rosetta 2*.

В заключении, выбор архитектуры *ARM* для исследования основано на сочетании энергоэффективности, многообразия аппаратных решений, широкого применения, доступности ресурсов, большого рынка и экономической эффективности.

Эта архитектура позволяет создавать эффективные и мощные вычислительные системы, что особенно важно в современном мире цифровых технологий. Более того, ее распространенность и популярность способствуют обширному сообществу разработчиков и обеспечивают доступность различных ресурсов и инструментов для работы.

Процессор *Apple M1*, как представитель этой архитектуры, является отличным выбором для демонстрации этих принципов. Его широкое использование, универсальность в поддержке операционных

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 *VirtualBox (Oracle VM VirtualBox)*

2.1.1 Структура и архитектура *VirtualBox*

VirtualBox представляет собой гипервизор с открытым исходным кодом, разрабатываемый *Oracle Corporation*. Его структура включает в себя следующие ключевые компоненты [9]:

1 *Hypervisor Core*: Ядро гипервизора выполняет основные функции виртуализации, управляя доступом виртуальных машин к физическим ресурсам хоста. Это включает в себя планирование ресурсов, управление памятью и обеспечение изоляции между виртуальными машинами. *Hypervisor Core* обеспечивает базовую абстракцию аппаратного обеспечения и реализует низкоуровневые операции виртуализации.

2 *Virtual Machine Monitor (VMM)*: *VMM* работает поверх ядра гипервизора и отвечает за конкретную реализацию виртуализации. Это включает в себя выполнение инструкций виртуальных машин, управление виртуальными процессорами и обеспечение поддержки виртуальных устройств. *VMM* является ключевым компонентом, обеспечивающим эффективное выполнение виртуальных машин на хостовом оборудовании.

3 *Frontends and Backends*: *Frontends* отвечают за взаимодействие с пользователем. Это может быть графический интерфейс для визуального управления виртуальными машинами или командная строка для управления через терминал. *Backends* обеспечивают связь между *Frontends* и *VMM*. Они принимают команды от *Frontends* и передают их в *VMM*, а затем передают результаты обратно *Frontends*.

4 *Device Emulation*: *VirtualBox* эмулирует аппаратное обеспечение, чтобы обеспечить совместимость с различными гостевыми операционными системами. Это включает в себя эмуляцию процессоров, сетевых устройств, звуковых карт и других устройств. *Device Emulation* обеспечивает стандартизированный интерфейс для взаимодействия виртуальных машин с виртуальным оборудованием.

Архитектура *VirtualBox* включает в себя следующие ключевые компоненты:

1 Виртуализация второго типа: *VirtualBox* представляет собой гипервизор второго типа, что означает, что он работает поверх операционной системы хоста. Это позволяет использовать общие драйвера и ресурсы хоста для более эффективной работы виртуальных машин.

2 Технологии виртуализации процессора: *VirtualBox* использует технологии виртуализации процессора, такие как *Intel VT-x* и *AMD-V*. Эти расширения позволяют гипервизору лучше управлять процессором и улучшают производительность виртуальных машин.

3 Поддержка различных архитектур: Гипервизор обеспечивает абстракцию аппаратного обеспечения, позволяя *VirtualBox* поддерживать различные архитектуры процессоров виртуальных машин, что включает в себя *x86*, *x86_64*, *ARM* и другие.

4 Переводчик инструкций: Гипервизор действует как переводчик между инструкциями, выполняемыми виртуальными машинами, и аппаратным обеспечением хоста. Это обеспечивает изоляцию между виртуальными машинами и позволяет им работать независимо друг от друга.

В целом, структура и архитектура *VirtualBox* сосредоточены на предоставлении эффективной виртуализации ресурсов и обеспечении удобного взаимодействия с пользователем.

2.1.2 История *VirtualBox*

VirtualBox, первоначально созданный немецкой компанией *Innotek GmbH*, вступил в мир виртуализации в 2007 году с выпуском своей первой публичной версии. *Innotek* разработала *VirtualBox* с учетом потребностей пользователей в мощном, гибком и бесплатном решении для виртуализации.

Важным моментом в истории *VirtualBox* стало приобретение *Innotek* компанией *Sun Microsystems* в 2008 году. Это поглощение открыло новые горизонты для развития *VirtualBox*, так как *Sun* была одной из ведущих компаний в области компьютерных технологий и имела значительный опыт в области программного обеспечения и серверных технологий.

Однако, история *VirtualBox* продолжила свое развитие после того, как *Sun Microsystems* сама была приобретена *Oracle Corporation* в 2010 году. Под крылом *Oracle*, *VirtualBox* продолжил свое развитие и стал частью обширного портфеля продуктов *Oracle*.

Важно отметить, что *VirtualBox* был выпущен под лицензией *GNU GPL* (*General Public License*). Это означает, что исходный код программы был открыт для общественности, что способствовало активной поддержке сообщества разработчиков и пользователей. Этот факт содействовал улучшению и расширению функционала *VirtualBox*.

История *VirtualBox* отражает эволюцию технологий виртуализации и демонстрирует его значимость в сфере *IT*. Сегодня *VirtualBox* продолжает оставаться популярным инструментом для виртуализации, предоставляя

пользователям удобный и эффективный способ запуска виртуальных машин на различных платформах.

2.1.3 Достоинства *VirtualBox*

VirtualBox является свободно распространяемым и программным обеспечением с открытым исходным кодом. Это означает, что любой может бесплатно загрузить, использовать и изменять исходный код *VirtualBox* в соответствии с условиями лицензии *GNU GPL*. Эта особенность открывает широкие возможности для пользователей адаптировать программу под свои уникальные потребности, вносить улучшения и распространять их в сообществе [10].

VirtualBox предоставляет впечатляющую совместимость с различными операционными системами в качестве гостевых ОС. Это включает в себя не только различные версии *Windows* и *Linux*, но также поддержку для *macos*, *Solaris*, *BSD* и других. Гибкость *VirtualBox* в выборе гостевых операционных систем делает его универсальным инструментом для тестирования и разработки приложений, а также для изучения различных платформ.

Один из существенных плюсов *VirtualBox* – это его интуитивно понятный графический интерфейс. Пользователи могут легко создавать и управлять виртуальными машинами через удобные меню и настройки. Это особенно важно для тех, кто только начинает знакомиться с виртуализацией. Кроме того, *VirtualBox* обеспечивает разнообразные опции настройки, позволяя опытным пользователям более тонко настраивать виртуальные машины.

VirtualBox активно использует технологии виртуализации процессора, такие как *Intel VT-x* и *AMD-V*. Эти расширения позволяют виртуальным машинам напрямую взаимодействовать с аппаратным обеспечением хоста, что улучшает производительность. В результате виртуальные машины могут эффективно использовать вычислительные ресурсы хост-системы, что особенно важно при выполнении ресурсоемких задач.

Эти преимущества делают *VirtualBox* популярным среди пользователей, начиная от обычных домашних пользователей, заканчивая опытными разработчиками и системными администраторами.

2.1.4 Обоснование выбора *VirtualBox*

Обоснование выбора платформы программного обеспечения *VirtualBox* для данного курсового проекта заключается в следующих факторах:

- 1 Открытый исходный код: Возможность изучения и адаптации

исходного кода *VirtualBox* предоставляет уникальную возможность настраивать гипервизор в соответствии с требованиями курсовой работы. Это особенно важно при необходимости поддержки различных архитектур процессоров.

2 Поддержка различных архитектур: *VirtualBox* предоставляет возможность создания виртуальных машин на различных архитектурах процессоров, включая *x86*, *x86_64*, *ARM* и другие. Это обеспечивает широкий выбор вариантов для тестирования и сравнения алгоритмов на разных процессорах. Гибкость *VirtualBox* в этом отношении позволяет максимально приблизить тестовые условия к реальным сценариям использования алгоритмов.

3 Бесплатность и простота использования: *VirtualBox* является бесплатным программным обеспечением, что делает его доступным для студентов и исследователей с ограниченным бюджетом. Простота использования графического интерфейса упрощает создание и управление виртуальными машинами, позволяя сосредоточиться на сравнении алгоритмов, а не на сложностях настройки гипервизора.

4 Богатые возможности настройки: *VirtualBox* предоставляет обширные возможности настройки виртуальных машин, включая параметры процессора, сетевых подключений, объема оперативной памяти и другие. Это позволяет точно регулировать условия тестирования для достижения максимально точных результатов при сравнении алгоритмов.

5 Активное сообщество пользователей: *VirtualBox* обладает активным сообществом пользователей и разработчиков. Это обеспечивает поддержку, обмен опытом и доступ к обширной базе знаний.

Общее сочетание открытого исходного кода, поддержки различных архитектур, бесплатности, простоты использования и возможности создания практически идентичного окружения для проведения тестирования процессоров разных архитектур делает *VirtualBox* идеальным выбором для проведения сравнительного анализа эффективности разработки алгоритмов.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

Современные информационные технологии требуют оптимизированных алгоритмов для достижения максимальной производительности и эффективности при выполнении вычислительных задач. Однако на пути разработчиков стоят разнообразные архитектуры процессоров, каждая из которых имеет свои уникальные характеристики и особенности. Это включает в себя архитектуры типа *CISC*, характеризующиеся богатым набором инструкций, архитектуры типа *RISC*, предоставляющие более ограниченный набор инструкций. Поэтому исследование эффективности реализации алгоритмов на различных архитектурах становится критически важным для современных разработчиков и исследователей. Понимание, как разные архитектуры взаимодействуют с алгоритмами, и способы их оптимизации могут значительно повысить производительность и эффективность программного обеспечения.

Развитие многоядерных процессоров в современных компьютерах и серверах стало нормой, и это направление интенсивно развивается. Многоядерные процессоры имеют два или более физических вычислительных ядра, работающих параллельно. Это означает, что они способны обрабатывать несколько потоков вычислений одновременно, что существенно увеличивает общую производительность. В современных операционных системах и приложениях поддержка многозадачности и параллельных вычислений является стандартом. Однако для максимальной эффективности использования многоядерных процессоров важно знать, как алгоритмы взаимодействуют с этими ядрами и как можно максимально распределить вычислительные нагрузки между ними. Важно понимать, какие алгоритмы могут быть разделены на независимые части, которые могут выполняться параллельно, и как это реализовать на практике. Максимальное использование ресурсов многоядерных процессоров также является важным аспектом. Эффективное управление вычислительными ресурсами, включая ядра процессора, кэш-память и оперативную память, имеет критическое значение для общей производительности системы.

Таким образом, в условиях постоянно растущей мощности многоядерных процессоров, оптимизация алгоритмов для различных архитектур становится важным аспектом разработки программного обеспечения. Эта тема актуальна, так как позволяет оптимизировать выполнение параллельных вычислений, эффективно использовать

вычислительные ресурсы и обеспечивать максимальную производительность в современных вычислительных системах.

Разнообразие архитектур в сфере вычислительных систем – ключевая черта современного информационного мира. Среди этого многообразия выделяются наиболее часто встречающиеся архитектуры *CISC* (*Complex Instruction Set Computer*) и *RISC* (*Reduced Instruction Set Computer*). Каждая из этих архитектур имеет свои особенности и специфику, которые определяют ее область применения и производительность.

Архитектура *CISC* (*Complex Instruction Set Computing*) характеризуется обширным набором инструкций, позволяющим выполнять сложные операции с минимальным количеством команд. Это делает *CISC*-архитектуру привлекательной для общего назначения и решения разнообразных задач [11].

Внутри семейства архитектур *x86*, *SSE* (*Streaming SIMD Extensions*) представляет собой расширение, которое дополняет возможности процессора в области векторных вычислений. *SSE* впервые появилось в 1999 году и было ориентировано на обработку сигналов и мультимедийных данных. Эти инструкции позволяют одной команде обрабатывать несколько элементов данных одновременно, что улучшает производительность в приложениях, где требуется обработка больших объемов данных [13].

Основные характеристики *SSE* включают в себя поддержку 128-битных регистров *XMM*, специальных для *SIMD* операций. *SSE* предоставляет разнообразные инструкции для работы с вещественными числами, целыми числами и другими данными в параллельном режиме. Эти инструкции включают операции сложения, вычитания, умножения, деления, а также различные операции над данными, такие как смешивание, перестановка и другие. Применение *SSE* широко распространено в областях, таких как обработка графики, видеокодирование, научные вычисления, а также в разработке эффективных алгоритмов, где параллельные вычисления могут существенно улучшить производительность [14].

Архитектура *RISC* (*Reduced Instruction Set Computing*), наоборот, характеризуется более ограниченным набором инструкций, что способствует упрощению дизайна процессора и повышению его производительности. *RISC*-архитектуры отлично подходят для выполнения базовых операций и вычислений с высокой скоростью, но могут потребовать большего числа инструкций для выполнения сложных операций [12].

В рамках архитектуры *ARM*, широко используется технология *NEON*, представляющая собой *SIMD*-расширение, аналогичное *SSE* в архитектуре *x86* [15]. Инструкции *NEON* предоставляют возможность выполнения

параллельных вычислений над векторами данных и обеспечивают ускоренную обработку мультимедийных данных. *NEON* включает в себя 64- и 128-битные регистры *SIMD*, а также обширный набор инструкций для работы с целыми и вещественными числами [16]. Эти инструкции позволяют эффективно выполнять операции сложения, умножения, смешивания и другие, что особенно полезно в областях, таких как обработка изображений, аудио кодирование, а также в области эффективной разработке алгоритмов.

В сравнении с *SSE*, *NEON* часто более эффективно использует энергию, что делает его привлекательным для мобильных устройств и встраиваемых систем.

Сравнительное исследование эффективности алгоритмов на разных архитектурах становится критически важным, так как оно позволяет выбирать наилучший вариант для конкретных задач. Знание особенностей каждой архитектуры и способов их использования может существенно повысить производительность и эффективность программного обеспечения.

Для эффективного решения задач, связанных с сравнением эффективности реализации алгоритмов на различных процессорных архитектурах используется язык программирования *C++*. Выбор *C++* обусловлен его высокой производительностью, широкими возможностями для системного программирования и оптимизации кода. Этот язык предоставляет разработчикам гибкий инструментарий для работы с памятью и многопоточностью, что особенно важно при реализации и адаптации алгоритмов под разнообразные процессорные архитектуры.

Для сравнения эффективности алгоритмов будет использована библиотека *<chrono>*. Она предоставляет механизмы для измерения времени выполнения кода и является важным инструментом в бенчмарках, профилировании и оптимизации программного обеспечения.

Библиотека *<chrono>* в *C++* является частью стандартной библиотеки и была введена в стандарте *C++11*. Ее основная цель – предоставить универсальный и точный механизм для работы с временем и измерения времени выполнения программ.

Преимущества библиотеки *<chrono>*:

- 1 Библиотека предоставляет типы данных для представления временных интервалов и моментов во времени, а также операции для работы с ними.

- 2 Позволяет работать с временем в различных часовых поясах и предоставляет функционал для конвертации времени между ними.

- 3 Абстрагирует код от различий в реализации времени на различных платформах, обеспечивая переносимость.

В дополнение к C++ используется гипервизор *VirtualBox*, который представляет собой мощное средство для создания виртуальных машин с целью уменьшить количество посторонних факторов, мешающих более четко и точно увидеть разницу в эффективности реализации алгоритмах на процессорах разных архитектур. Такой подход гарантирует надежность и предсказуемость экспериментов, исключая факторы, связанные с физическими вычислительными системами.

Сочетание языка программирования C++ с использованием гипервизора *VirtualBox* формирует идеальное окружение для проведения исследований и сравнительного анализа производительности алгоритмов на различных процессорных архитектурах. Это обусловлено несколькими ключевыми факторами.

Во-первых, использование C++ обеспечивает высокую производительность благодаря своей компилируемой природе и богатым возможностям языка. Это позволяет разработчикам эффективно реализовывать и тестировать алгоритмы, а также использовать *SIMD*-инструкции, такие как *SSE* и *NEON*, для оптимизации параллельных вычислений.

Во-вторых, применение *SIMD*-инструкций, таких как *SSE* и *NEON*, является ключевым аспектом при анализе производительности алгоритмов. Эти инструкции обеспечивают параллельную обработку данных, что может значительно ускорить выполнение операций, особенно в контексте больших объемов данных, например, в мультимедийных приложениях.

Гипервизор *VirtualBox* дает возможность создавать изолированные виртуальные среды для тестирования на различных процессорных архитектурах. Это значительно упрощает процесс сравнительного анализа, позволяя исследователям оценивать производительность алгоритмов в различных условиях.

Такое сочетание технологий позволяет более глубоко понимать влияние различных процессорных архитектур на выполнение алгоритмов. Результаты таких исследований могут быть ценными при выборе оптимальных алгоритмов для конкретных задач и условий эксплуатации, а также это позволит более глубоко понять, как различные архитектуры процессора влияют на выполнение алгоритмов, и сделать обоснованные выводы о преимуществах и недостатках каждой из них.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

4.1 Выбор подходящих алгоритмов

При выборе алгоритмов для исследования эффективности на процессорах разных архитектур, *AMD Ryzen 7 5800H (x86)* и *Apple M1 (ARM)*, было принято решение сосредоточиться на алгоритмах быстрого преобразования Фурье (*FFT*) и расстояния Левенштейна.

Алгоритм расстояния Левенштейна был включен в исследование производительности на процессорах разных архитектур по ряду ключевых причин, делающих его важным объектом анализа:

В первую очередь, алгоритм Левенштейна представляет собой метрику редакторского расстояния, измеряя минимальное количество редакторских операций (вставка, удаление, замена) для преобразования одной строки в другую. Это имеет непосредственное применение в областях сравнения текстов, исправления ошибок в строках и других задачах редактирования текста, что делает его актуальным для анализа в контексте реальных приложений.

Включение алгоритма Левенштейна также обусловлено его высокой универсальностью и широким спектром применения в различных областях, от поисковых систем до реализации автокоррекции и редакторов текста.

Особый интерес представляет аспект использования *SIMD*-инструкций, таких как *SSE* для *x86* и *NEON* для *ARM*, в реализации алгоритма Левенштейна. Поскольку этот алгоритм поддается векторизации, использование *SIMD*-инструкций может существенно ускорить процесс его выполнения. Этот аспект становится критически важным в контексте исследования производительности различных процессорных архитектур и позволяет более глубоко понять, как *SIMD*-расширения влияют на эффективность выполнения сложных текстовых алгоритмов.

Алгоритм быстрого преобразования Фурье (*FFT*) был включен в исследование производительности на процессорах различных архитектур по нескольким ключевым обоснованиям:

В первую очередь, алгоритм *FFT* является широко применяемым алгоритмом, используемым в различных областях, включая обработку сигналов, мультимедийные приложения и телекоммуникации. Такой выбор позволяет оценить производительность процессоров на конкретных вычислениях, активно используемых в реальных приложениях.

Дополнительно, алгоритм *FFT* обладает высокой вычислительной сложностью, особенно при обработке больших объемов данных. Это обеспечивает благоприятные условия для проведения сравнительного анализа производительности процессоров разных архитектур, поскольку *FFT* представляет типичную вычислительную нагрузку, с которой современные процессоры должны справляться эффективно.

Одним из ключевых моментов включения алгоритма *FFT* в исследование является также его потенциал для использования *SIMD*-инструкций, таких как *SSE* для *x86* и *NEON* для *ARM*. Поскольку *FFT* может эффективно разделять вычисления по разным элементам вектора данных, использование *SIMD*-расширений позволяет параллельно обрабатывать несколько элементов данных, что существенно ускоряет выполнение алгоритма.

Таким образом, выбор алгоритмов и использование *SIMD*-инструкций *SSE* и *NEON* обоснованы не только широким применением данных алгоритмов в реальных задачах, но и стремлением к максимальному выигрышу в производительности на выбранных процессорах разных архитектур.

4.2 Расстояние Левенштейна

4.2.1 Основные сведения

Расстояние Левенштейна (редакционное расстояние, дистанция редактирования) — метрика, измеряющая по модулю разность между двумя последовательностями символов. Она определяется как минимальное количество односимвольных операций (а именно вставки, удаления, замены), необходимых для превращения одной последовательности символов в другую. В общем случае, операциям, используемым в этом преобразовании, можно назначить разные цены. Широко используется в теории информации и компьютерной лингвистике [17].

Впервые задачу поставил в 1965 году советский математик Владимир Левенштейн. Впоследствии более общую задачу для произвольного алфавита связали с его именем. Так же большой вклад в изучение вопроса внёс Дэн Гасфилд.

Расстояние Левенштейна и его обобщения активно применяется для исправления ошибок в слове (в поисковых системах, базах данных, при вводе текста, при автоматическом распознавании отсканированного текста или речи), для сравнения текстовых файлов, а также широко используется в биоинформатике для сравнения генов, хромосом и белков.

Блок-схема алгоритма программы представлена в приложении В.

Описание блок-схемы алгоритма Левенштейна:

1 Инициализация: Создание матрицы, где каждая ячейка $[i, j]$ представляет собой минимальное расстояние между первыми i символами первой строки и первыми j символами второй строки.

2 Заполнение матрицы: Построение матрицы пошагово, начиная с верхнего левого угла и заканчивая в правом нижнем углу. На каждом шаге вычисляется минимальное расстояние до текущей ячейки, исходя из значений в соседних ячейках.

Это выполняется с использованием формулы:

$$D[i, j] = \min(D[i - 1, j] + 1, D[i, j - 1] + 1, D[i - 1, j - 1] + \delta_{ij}), \quad (2)$$

где, δ_{ij} равно 0, если символы в позициях i и j равны, и 1 в противном случае.

3 Расстояние Левенштейна: Расстояние между строками находится в правом нижнем углу матрицы и представляет минимальное количество редакционных операций.

Алгоритм Левенштейна (рисунок 4.1) относится к классу динамического программирования.

Динамическое программирование — это метод решения задач, которые могут быть разбиты на подзадачи, и в которых результат каждой подзадачи сохраняется и может быть использован для решения более крупной задачи.

		P	O	L	Y	N	O	M	I	A	L
	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
T	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	7	6	6	6	6	7	8
A	10	9	8	8	8	7	7	7	7	6	7
L	11	10	9	8	9	8	8	8	8	7	6

Рисунок 4.1 – Пример работы алгоритма Левенштейна

Асимптотическая сложность алгоритма Левенштейна с использованием

динамического программирования составляет $O(n \cdot m)$, где m – длина первой строки, а n – длина второй строки [18].

Достоинства:

1 Алгоритм может быть применен для решения широкого спектра задач, таких как исправление опечаток, сравнение строк, поиск наилучшего соответствия и другие, где требуется измерение схожести между последовательностями символов.

2 Несмотря на квадратичную асимптотическую сложность, алгоритм часто оказывается достаточно эффективным на практике, особенно при коротких или схожих строках.

3 Алгоритм Левенштейна основан на простых принципах вставки, удаления и замены символов, что делает его понятным и доступным для понимания.

Недостатки:

1 Алгоритм Левенштейна не учитывает контекст или смысл слов. Он оперирует символами, не понимая их значения, что может привести к недостаточно точным результатам в некоторых контекстах, например, при исправлении ошибок в словах с несколькими возможными вариантами исправления.

2 Алгоритм не обрабатывает транспозиции (перестановку символов), требуя двух операций (удаление и вставка) вместо одной.

3 Асимптотическая сложность алгоритма $O(n \cdot m)$, делает его неэффективным для больших строк. При увеличении длины строк время выполнения может значительно увеличиваться.

4.2.2 Эффективная реализация алгоритма на архитектуре x86

Общая идея эффективности реализации алгоритма Левенштейна на архитектуре x86 заключается в двух ключевых аспектах – это использовании *SIMD*-инструкций и управлении выравниванием данных.

Использование *SIMD* (*Single Instruction, Multiple Data*) является ключевым фактором для ускорения алгоритма. *SSE*-регистры позволяют параллельно обрабатывать несколько элементов данных, что особенно полезно в случае обработки строк, где операции часто могут быть выполнены над несколькими символами одновременно. Это повышает эффективность выполнения операций вставки, удаления и замены символов, уменьшая количество итераций циклов и ускоряя общий процесс.

Выравнивание данных также играет важную роль. Использование `_mm_loadu_si128` позволяет обрабатывать не выровненные данные, что

удобно, когда строки не выровнены по определенным границам в памяти. Не выровненные загрузки позволяют более гибко обрабатывать данные, но в определенных ситуациях, например, при работе с крупными объемами данных, может потребоваться выравнивание для достижения максимальной производительности.

Выравнивание обеспечивает более эффективный доступ к данным в регистрах *SIMD* и снижает накладные расходы при выполнении операций.

Использование различных *SIMD*-инструкций в реализации алгоритма Левенштейна на архитектуре *x86* с использованием *SSE* предоставляет эффективные средства для выполнения операций с векторами целых чисел.

Используемые *SSE* инструкции (листинг А.2):

1 Сравнение (*_mm_cmpeq_epi32*): Данная инструкция выполняет операцию сравнения поэлементно для двух 128-битных векторов. В контексте алгоритма Левенштейна она используется для сравнения символов в двух строках. Результат сравнения будет вектором, в котором каждый элемент равен 1, если соответствующие элементы векторов равны, и 0 в противном случае.

2 Сложение (*_mm_add_epi32*): Инструкция сложения применяется к векторам целых чисел, выполняя сложение поэлементно. Используется для вычисления стоимости операций вставки, удаления и замены символов, представленных векторами.

3 Вычитание (*_mm_sub_epi32*): Эта инструкция применяется к векторам целых чисел и выполняет операцию вычитания поэлементно. В алгоритме Левенштейна используется для вычисления стоимости операции замены символов.

4 Минимум (*_mm_min_epi32*): Инструкция находит минимальное значение между соответствующими элементами двух векторов. Применяется для выбора минимальной стоимости из операций вставки, удаления и замены символов.

5 Загрузка и сохранение (*_mm_loadu_si128*, *_mm_store_si128*): Инструкции загрузки и сохранения позволяют эффективно перемещать данные между регистрами *SIMD* и памятью. Инструкция *_mm_loadu_si128* используется для загрузки 128-битных значений из не выровненной области памяти, что обеспечивает гибкость в обработке данных различной выравненности. Инструкция *_mm_store_si128* используется для сохранения 128-битных значений в памяти.

Использование *SIMD*-инструкций и управление выравниванием данных в реализации алгоритма Левенштейна на архитектуре *x86* с использованием

технологии *SSE* обеспечивают значительное повышение эффективности.

Параллельная обработка нескольких элементов данных за одну инструкцию и гибкость в обработке не выровненных данных позволяют сократить количество итераций циклов и уменьшить накладные расходы, связанные с управлением циклом и копированием данных.

Использование инструкций сравнения, сложения, вычитания, и нахождения минимума позволяет эффективно вычислять стоимость операций между векторами целых чисел, представляющими символы строк. Эти инструкции обеспечивают точный и параллельный расчет стоимости вставки, удаления и замены символов.

Оптимизированный обмен строками через указатели способствует уменьшению затрат на копирование данных, что важно для обеспечения эффективного доступа к памяти.

Таким образом, эти техники в совокупности делают реализацию алгоритма Левенштейна на архитектуре *x86* эффективным инструментом для быстрого вычисления редакционного расстояния между строками.

4.2.3 Эффективная реализация алгоритма на архитектуре *ARM*

Общая идея эффективности реализации алгоритма Левенштейна на архитектуре *ARM* заключается в двух ключевых аспектах – это использование матрицы для хранения промежуточных результатов в алгоритме и векторизация с использованием *NEON* инструкций.

Использование матрицы *dp* представляет собой эффективную стратегию для хранения промежуточных результатов в алгоритме Левенштейна. Эта матрица служит для отслеживания стоимости редакционных операций между всеми возможными парами префиксов строк. Заполнение матрицы происходит пошагово, где каждый элемент $dp[i][j]$ представляет собой минимальную стоимость превращения первых i символов строки *str1* в первые j символов строки *str2*.

Использование матрицы *dp* позволяет избежать повторных вычислений, поскольку результаты предыдущих операций сохраняются для дальнейшего использования.

Векторизация с использованием *NEON* инструкций предоставляет возможность параллельной обработки нескольких элементов данных одновременно. *NEON* инструкции применяются для выполнения арифметических операций над векторами данных, что ускоряет выполнение операций сложения, вычитания и сравнения. Например, операции сложения и сравнения выполняются параллельно для четырех элементов, что

обеспечивает эффективную обработку блоков данных и повышение производительности.

Параллельная обработка строк означает, что расстояние между символами вычисляется с использованием векторизованных инструкций для каждого символа одновременно. Это позволяет уменьшить количество итераций цикла, так как каждая итерация обрабатывает несколько символов сразу. Такой подход особенно полезен в контексте строковых операций, где множество вычислений может быть выполнено параллельно.

Используемые *NEON* инструкции (листинг А.3):

1 Инструкция *vld1q_s32* используется для эффективной загрузки данных из памяти в регистр вектора. Она применяется для загрузки значений из матрицы *dp*, представляющих собой стоимость редакционных операций. Загрузка векторов целых чисел происходит одновременно, что способствует увеличению пропускной способности памяти и ускорению процесса.

2 Инструкция *vst1q_s32* используется для эффективной записи данных из регистра вектора обратно в память. Она применяется для сохранения результатов операций в матрицу *dp*. Эффективное сохранение векторов данных обеспечивает оптимальное использование памяти и ускоряет процесс записи результатов.

3 Инструкция *vaddq_s32* выполняет операцию сложения поэлементно для двух векторов целых чисел. Она используется для добавления константы (например, 1) к векторам, представляющим стоимость операций вставки и удаления символов. Параллельное сложение элементов вектора ускоряет выполнение арифметических операций.

4 Инструкция *vminq_s32* находит минимальное значение между соответствующими элементами двух векторов. Она применяется для выбора минимальной стоимости из операций вставки, удаления и замены символов. Параллельная обработка элементов вектора ускоряет поиск минимума.

5 Инструкция *vseqq_s32* выполняет операцию условного сравнения поэлементно для двух векторов целых чисел. Она применяется для сравнения символов двух строк. Результат сравнения (вектор булевых значений) используется для коррекции стоимости операции замены. Так, если символы равны, к стоимости замены прибавляется 0, иначе прибавляется 1. Условное сравнение позволяет эффективно интегрировать условия векторного выполнения операций.

Таким образом, векторизация, параллельная обработка строк и оптимизированный доступ к данным с использованием *NEON* инструкций в совокупности создают эффективную реализацию алгоритма Левенштейна для

архитектуры ARM.

4.3 Быстрое преобразование Фурье

4.3.1 Основные сведения

Быстрое преобразование Фурье (БПФ) – это эффективный алгоритм для вычисления дискретного преобразования Фурье (ДПФ) – математической процедуры, которая преобразует временной сигнал в его представление в частотной области [19].

ДПФ определяется по формуле:

$$X_k = \sum_{m=0}^{n-1} x_m \cdot e^{-i2\pi km/n}, \quad (3)$$

где $k = 0, \dots, n-1$ и $e^{-i2\pi k/n}$ – примитивный корень n -й степени из 1.

Алгоритм был предложен Джеймсом Кули и Джоном Тьюки в 1965 году.

ДПФ и его эффективная форма БПФ – играют ключевую роль в различных областях науки и техники, таких как:

1 Сигнальная обработка: В сигнальной обработке ДПФ используется для анализа частотного содержания сигналов. Это позволяет исследовать, какие частоты присутствуют в сигнале, и какие амплитуды у этих частот.

2 Телекоммуникации: В телекоммуникациях ДПФ применяется для многоканальной передачи данных.

3 Обработка изображений: В обработке изображений ДПФ используется для анализа частотных характеристик изображений.

4 Алгоритмы свертки: ДПФ применяется в алгоритмах свертки для ускорения вычислений.

Блок-схема алгоритма программы представлена в приложении В.

Описание блок-схемы алгоритма БПФ:

1 Основная идея БПФ связана с использованием периодичности комплексных экспонент. Для любого целого числа N , экспонента $e^{-j(2\pi/N)kn}$ имеет период N . Таким образом, значения комплексных экспонент для различных k могут повторяться, что создает возможность оптимизации.

2 БПФ начинается с разбиения ДПФ на две части: четные и нечетные компоненты. Это основано на том, что четные и нечетные элементы в сумме в исходной формуле ДПФ могут быть обработаны независимо.

3 Каждую из полученных подзадач для четных и нечетных компонент алгоритм БПФ применяет рекурсивно. Таким образом, задача разбивается на более мелкие и подзадачи меньшего размера.

4 После рекурсивной обработки четных и нечетных компонент, результаты комбинируются для получения окончательного результата. Этот шаг выполняется с использованием периодичности комплексных экспонент, что позволяет объединить результаты подзадач в общий результат.

5 БПФ значительно сокращает количество операций по сравнению с прямым вычислением ДПФ. Вместо N^2 операций, как в случае прямого вычисления, БПФ требует только $N \log N$ операций, что делает его более эффективным для больших N .

Основываясь на подходе "разделяй и властвуй" (рисунок 4.2), сложность по времени алгоритма БПФ Кули-Тьюки оценивается как $O(N \log N)$, что делает его предпочтительным для больших объемов данных. Общая идея БПФ заключается в разбиении задачи вычисления ДПФ на более мелкие и более управляемые подзадачи, с последующим объединением результатов. Этот подход является ключевым для достижения высокой эффективности алгоритма в сравнении с более прямыми методами вычисления ДПФ [20].

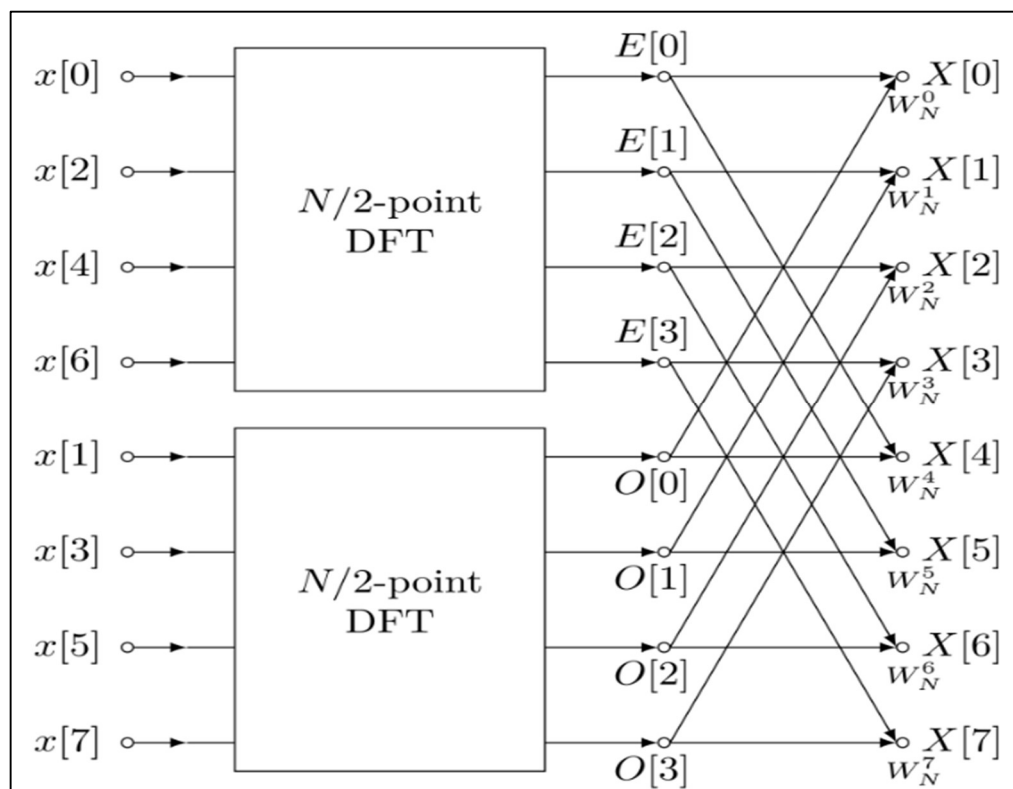


Рисунок 4.2 – Пример структуры алгоритма БПФ

Достоинства БПФ:

1 БПФ значительно ускоряет процесс вычисления ДПФ по сравнению с прямым вычислением. За счет использования стратегии "разделяй и властвуй" и оптимизаций, алгоритм Кули-Тьюки снижает сложность вычислений с $O(N^2)$ до $O(N \log N)$.

2 БПФ применим к разнообразным типам данных и сигналов. Это означает, что его можно использовать в различных областях, включая сигнальную обработку, телекоммуникации, обработку изображений, анализ данных и многие другие.

Недостатки БПФ:

1 Ограничение на размер данных: Некоторые реализации БПФ могут сталкиваться с ограничениями по памяти или размеру входных данных. В частности, стандартный алгоритм Кули-Тьюки требует дополнительной памяти для хранения промежуточных результатов, что может стать проблемой при работе с большими объемами данных.

В целом, быстрое преобразование Фурье остается мощным инструментом для обработки сигналов и анализа частотных характеристик различных типов данных, и его преимущества обычно превалируют над недостатками, особенно при работе с крупными объемами данных.

4.3.2 Эффективная реализация алгоритма на архитектуре x86

Общая идея эффективности реализации алгоритма БПФ на архитектуре x86 заключается в использовании SSE-инструкций.

SSE (*Streaming SIMD Extensions*) инструкции позволяют обрабатывать несколько элементов данных за одну инструкцию. В FFT каждый этап вычислений, включая бабочки (*butterfly*) и тригонометрические операции, может быть векторизован с использованием регистров XMM, которые могут содержать несколько элементов *double*. Это приводит к одновременной обработке нескольких значений и уменьшению общего числа операций.

В контексте алгоритма быстрого преобразования Фурье (БПФ), термины "бабочка" и "twiddle-фактор" являются ключевыми элементами, описывающими операции, выполняемые в процессе преобразования.

Бабочка (*Butterfly*) – это основной элемент операции в алгоритме БПФ. Операция бабочки представляет собой комбинацию умножения и сложения двух комплексных чисел. Существует два типа бабочек: бабочки на "верхнем" этапе (*Top Butterfly*) и бабочки на "нижнем" этапе (*Bottom Butterfly*). На каждом этапе БПФ, комплексные числа проходят через бабочки, обеспечивая последовательное преобразование данных в частотной области.

Top Butterfly: на верхнем этапе бабочки соединяют два комплексных числа из различных половин вектора, производя умножение на *twiddle*-фактор и сложение. Это осуществляется в два этапа.

Bottom Butterfly: на нижнем этапе бабочки соединяют два комплексных числа из одной половины вектора, снова с умножением на *twiddle*-фактор и сложением.

Twiddle-фактор – это комплексное число, представляющее поворот вокруг начала координат на единичном круге в комплексной плоскости. В контексте БПФ, *twiddle*-факторы используются для множителей при умножении

Эффективность отдельных *SSE* инструкций в реализации *FFT* (листинг А.5):

1 *_mm_load_pd*: Эта инструкция загружает два значения с памяти в регистр *XMM*. Применительно к *FFT*, она эффективно загружает два значения (действительную и мнимую части) из массива данных в регистр, минимизируя задержки ввода-вывода и улучшая пропускную способность памяти.

2 *_mm_store_pd*: Инструкция выполняет обратную операцию – сохранение двух значений из регистра *XMM* в память. В *FFT* используется для записи результатов обработки обратно в массив данных.

3 *_mm_mul_pd*: Эта инструкция производит умножение двух значений, содержащихся в регистре *XMM*. В контексте *FFT*, где часто требуется умножение на комплексные числа (*twiddle*-факторы), *_mm_mul_pd* позволяет выполнять параллельное умножение двух пар значений в одном регистре, что значительно уменьшает количество инструкций и улучшает производительность.

4 *_mm_add_pd*: Эта инструкция служит для сложения двух значений в регистре *XMM*. В *FFT* применяется для параллельного сложения действительной и мнимой частей комплексных чисел после умножения на *twiddle*-факторы. Использование *_mm_add_pd* позволяет эффективно выполнить сложение векторов в одном регистре, ускоряя алгоритм.

5 *Trigonometric Functions (cos и sin)*: Функции *cos* и *sin* используются для предварительного вычисления значений *twiddle*-факторов, которые затем используются в бабочках *FFT*. Предварительные вычисления *twiddle*-факторов снижают количество операций в цикле, так как сохраняется заранее вычисленное значение, и происходит обращение к нему при каждой итерации цикла. Это существенно ускоряет процесс *FFT*.

Использование этих *SSE* инструкций позволяет достичь высокой эффективности выполнения *FFT* на архитектуре *x86*. Применение

векторизации данных и параллельных операций существенно уменьшает количество инструкций и ускоряет выполнение алгоритма за счет максимального использования возможностей *SIMD*.

4.3.3 Эффективная реализация алгоритма на архитектуре *ARM*

Алгоритм *FFT* на архитектуре *ARM* с использованием *NEON* инструкций направлен на максимизацию параллелизма и векторизации данных.

Общая структура алгоритма соответствует принципам *FFT*, но его выполнение оптимизировано для работы с *SIMD* возможностями *NEON*. Вложенные циклы позволяют эффективно использовать параллелизм на различных уровнях.

Используемые *NEON* инструкции (листинг А.6):

1 *vld1_f32*: Инструкция загружает значения из памяти в векторный регистр. В данной реализации используется для эффективной загрузки действительных и мнимых частей комплексных чисел из массива векторов в регистр *NEON*.

2 *vst1_f32*: Инструкция сохраняет значения из векторного регистра в память. В данной реализации используется для эффективной записи результатов вычислений обратно в массив векторов.

3 *vmul_f32*: Инструкция выполняет умножение двух значений, содержащихся в векторных регистрах. В контексте *FFT* она применяется для параллельного умножения действительной и мнимой частей комплексных чисел на *twiddle*-факторы.

4 *vadd_f32*: Инструкция служит для сложения двух значений, содержащихся в векторных регистрах. В *FFT*, она применяется для параллельного сложения действительной и мнимой частей комплексных чисел после умножения на *twiddle*-факторы.

5 *vcreate_f32*: Инструкция используется для создания векторов на основе значений, предоставляемых функциями *cos* и *sin*. В данной реализации она служит для предварительного вычисления *twiddle*-факторов.

Таким образом, эти *NEON* инструкции обеспечивают оптимизированный доступ к данным, векторизацию умножения и сложения, а также предварительное вычисление *twiddle*-факторов, что совместно способствует высокой эффективности реализации *FFT* на архитектуре *ARM* с использованием *NEON* инструкций.

5 СРАВНЕНИЕ ЭФФЕКТИВНОСТИ АЛГОРИТМОВ

5.1 Основные сведения

Функциональная схема программы представлена в приложении Б.

В рамках разработки функциональной схемы были выделены четыре ключевых блока:

1 Инициализация начальных значений. На данном этапе происходит создание объекта, необходимого для хранения результатов тестирования, и инициализация различных входных данных для алгоритмов.

2 Тестирование алгоритмов. В этом блоке происходит измерение времени выполнения алгоритма, а также сохранение этой информации в ранее созданный объект.

3 Вывод результатов в консоль. Здесь производится вывод результатов предыдущего этапа в консоль в формате таблицы, содержащей различные заголовки времени выполнения и количества входных данных для каждого из алгоритмов.

4 Запись результатов в файл – занесенные на втором этапе результаты тестирования записываются в файл в формате *json* для дальнейшего анализа.

Для более детального анализа эффективности алгоритмов используется библиотека *<chrono>*, предназначенная для измерения времени выполнения программы с высокой точностью.

Дополнительно, для минимизации погрешностей, связанных с внешними факторами и различиями в комплектующих систем, в проведении тестирования эффективности алгоритмов используется виртуализация среды с помощью *VirtualBox*. Этот инструмент позволяет создавать виртуальные машины с однородной конфигурацией, что обеспечивает равные условия для проведения тестов.

Таким образом, использование *VirtualBox* в комбинации с библиотекой *<chrono>* гарантирует надежность и сопоставимость результатов анализа эффективности алгоритмов на разных архитектурах.

5.2 Анализ эффективности алгоритма Левенштейна

Результат работы программы (рисунок 5.1), предоставляет информацию о времени выполнения различных реализаций алгоритма Левенштейна с использованием инструкций *SSE* и без них в микросекундах при постоянно увеличивающемся количестве входных данных, а также, отношение времени

выполнения на архитектуре *x86*.

Size	Levenshtein_SSE [mcs]	Levenshtein [mcs]	L/L_SSE
100	1	18	18.00
1000100	17562	24886	1.42
2000100	42560	51901	1.22
3000100	54842	78332	1.43
4000100	72181	114960	1.59
5000100	92180	136728	1.48
6000100	102394	167774	1.64
7000100	140961	190591	1.35
8000100	145345	215395	1.48
9000100	173050	257917	1.49

Рисунок 5.1 – Результат работы алгоритма Левенштейна на архитектуре *x86*

Результат работы программы (рисунок 5.2), предоставляет информацию о времени выполнения различных реализаций алгоритма Левенштейна с использованием инструкций *NEON* и без них в микросекундах при постоянно увеличивающемся количестве входных данных, а также, отношение времени выполнения на архитектуре *ARM*.

Size	Levenshtein_NEON [mcs]	Levenshtein [mcs]	L/L_NEON
100	229	4	0.02
1000100	44453	21167	0.48
2000100	40259	35113	0.87
3000100	55994	60000	1.07
4000100	85298	83408	0.98
5000100	82936	91211	1.10
6000100	110859	94614	0.85
7000100	119286	213476	1.79
8000100	170657	162313	0.95
9000100	194307	186942	0.96

Рисунок 5.2 – Результат работы алгоритма Левенштейна на архитектуре *ARM*

В ходе анализа эффективности реализаций алгоритма Левенштейна на различных архитектурах, можно сделать вывод о том, что реализация с

использованием *SSE* инструкций на архитектуре *x86* показала стабильное и значительное ускорение выполнения по сравнению с обычной реализацией на различных объемах данных, ускорение на небольших объемах данных достигает 18 раз, а в среднем остается значительным, примерно 1.5 раза, на более крупных данных.

С другой стороны, реализация с использованием *NEON* инструкций на архитектуре *ARM* проявила переменную эффективность. Эффективность *NEON* инструкций не всегда соответствует ожиданиям, особенно на больших объемах данных, где ускорение становится менее существенным по сравнению с реализацией без *NEON* инструкций.

Таким образом, исходя из предоставленных данных, в соответствии с рисунком 1 приложения Д, сравнений об общей эффективности, можно сделать вывод, что реализация алгоритма Левенштейна с использованием определенных инструкций, заточенных под особенности архитектуры процессора (*SSE* для *x86* и *NEON* для *ARM*), дает значительный прирост в скорости выполнения алгоритма по сравнению с его обычной реализацией.

Однако, важно отметить, что при сравнении *SSE* и *NEON* реализаций на различных архитектурах (*x86* и *ARM*) не обнаружено значительных различий в эффективности. Обе реализации показывают впечатляющие результаты и в среднем равны между собой для данной реализации алгоритма.

Важным аспектом является понимание того, что обе архитектуры процессоров, *CISC* (*Complex Instruction Set Computing*) и *RISC* (*Reduced Instruction Set Computing*), в современных реализациях в значительной степени сближаются в плане использования современных технологий и инструкций.

Тенденция к переплетению обеих архитектур заключается в стремлении интегрировать лучшие аспекты из каждой системы с целью обеспечения более высокой производительности и эффективности обработки данных.

Таким образом, в современных реализациях *CISC* и *RISC* встроены механизмы, которые позволяют эффективно использовать передовые технологии и оптимизированные инструкции, такие как *SSE* и *NEON*. Это позволяет достичь сопоставимых результатов в ускорении выполнения алгоритма Левенштейна независимо от выбранной архитектуры процессора.

5.3 Анализ эффективности алгоритма БПФ

Результат работы программы (рисунок 5.3), предоставляет информацию о времени выполнения различных реализаций алгоритма быстрого преобразования Фурье (БПФ) с использованием инструкций *SSE* и без них в

микросекундах при постоянно увеличивающемся количестве входных данных, а также, отношение времени выполнения на архитектуре *x86*.

Size	FFT_SSE [mcs]	FFT [mcs]	FFT/ FFT_SSE
1024	1161	10337	8.90
2048	2276	18743	8.24
4096	5177	42553	8.22
8192	10669	81983	7.68
16384	20115	173154	8.61
32768	41583	345913	8.32
65536	91687	736618	8.03
131072	195971	1495134	7.63
262144	410785	3094393	7.53

Рисунок 5.3 – Результат работы алгоритма БПФ на архитектуре *x86*

Результат работы программы (рисунок 5.4), предоставляет информацию о времени выполнения различных реализаций алгоритма БПФ с использованием инструкций *NEON* и без них в микросекундах при постоянно увеличивающемся количестве входных данных, а также, отношение времени выполнения на архитектуре *ARM*.

Size	FFT_NEON [mcs]	FFT [mcs]	FFT/ FFT_NEON
1024	1138	19714	17.32
2048	1359	22147	16.30
4096	4959	30828	6.22
8192	5698	63256	11.10
16384	11902	119274	10.02
32768	23572	240055	10.18
65536	49056	493963	10.07
131072	103602	1048813	10.12
262144	237693	2069204	8.71

Рисунок 5.4 – Результат работы алгоритма БПФ на архитектуре *ARM*

В ходе анализа эффективности реализаций алгоритма БПФ на различных архитектурах, можно сделать вывод о том, что реализация с использованием *SSE* инструкций на архитектуре *x86* показала стабильное и значительное ускорение выполнения по сравнению с обычной реализацией на различных объемах данных, ускорение в среднем остается значительным, примерно 7-8 раз.

С другой стороны, реализация с использованием *NEON* инструкций на архитектуре *ARM* проявила переменную эффективность. Эффективность использования *NEON* инструкций в среднем случае примерно в 10 раз быстрее, чем выполнения по сравнению с обычной реализацией, но иногда можно заметить небольшие просадки до 6-8 раз.

Таким образом, исходя из предоставленных данных, в соответствии с рисунком 2 приложения Д, можно сделать вывод, что реализация алгоритма БПФ с использованием определенных инструкций, заточенных под особенности архитектуры процессора (*SSE* для *x86* и *NEON* для *ARM*), дает значительный прирост в скорости выполнения алгоритма по сравнению с его обычной реализацией.

Однако, важно отметить, что при сравнении *SSE* и *NEON* реализаций на различных архитектурах (*x86* и *ARM*) не обнаружено значительных различий в эффективности. Обе реализации показывают впечатляющие результаты и в среднем равны между собой для данной реализации алгоритма.

Необходимо отметить, что как архитектуры процессоров *CISC* (*Complex Instruction Set Computing*), так и *RISC* (*Reduced Instruction Set Computing*) в настоящее время предоставляют широкие возможности для использования оптимизированных инструкций. Оба подхода позволяют программистам в полной мере воспользоваться преимуществами современных технологий, таких как *SIMD* (*Single Instruction, Multiple Data*), что способствует более эффективному выполнению алгоритма БПФ (быстрое преобразование Фурье).

ЗАКЛЮЧЕНИЕ

В рамках данного курсового проекта были выполнены все поставленные цели и проведено исследование с целью разработки эффективных реализаций алгоритмов – Левенштейна и Быстрого преобразования Фурье (БПФ) – для применения на различных архитектурах процессоров, таких как, *x86* и *ARM*.

Цель исследования включала в себя глубокий анализ архитектур *x86* и *ARM* с особенным вниманием к их инструкционным наборам, таким как, инструкции *SSE* для архитектуры *x86* и инструкции *NEON* для архитектуры *ARM*.

Разработанные алгоритмы Левенштейна и быстрого преобразования Фурье были активно адаптированы под уникальные архитектурные особенности процессоров *AMD Ryzen 7 5800h* и *Apple M1*, с учетом стратегии выполнения инструкций, характерной для *CISC* и *RISC* архитектуры.

Для обеспечения надежности результатов тестирования и исключения возможных искажений, связанных с аппаратными различиями, использовалась виртуализация при помощи *VirtualBox*. Это обеспечило создание виртуальных машин с однородной конфигурацией, предоставляя тем самым равные условия для проведения тестов на обеих архитектурах.

Основное внимание уделялось анализу эффективности алгоритмов на различных этапах их выполнения, с использованием библиотеки *<chrono>* для точного измерения времени выполнения.

Результаты тестирования демонстрировали значительное ускорение выполнения алгоритмов при использовании оптимизированных инструкций *SSE* и *NEON* на *x86* и *ARM* соответственно.

В частности, были проанализированы реализации алгоритма Левенштейна и БПФ с применением *SSE* и *NEON* инструкций.

Использование *SSE* инструкций на *x86* архитектуре показало стабильное и существенное улучшение производительности, тогда как на *ARM* с использованием *NEON* наблюдалась переменная эффективность в зависимости от объема данных.

Важным выводом является отсутствие существенных различий в эффективности *SSE* и *NEON* реализаций на разных архитектурах. Это говорит о том, что, несмотря на исторические различия между *CISC* и *RISC*, современные реализации обеих архитектур сближаются в использовании передовых технологий.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] Доманов, А. Т. Стандарт предприятия / А. Т. Доманов, Н. И. Сорока. – Минск: БГУИР, 2017. – 167 с.
- [2] Современные десктопные процессоры архитектуры x86 [Электронный ресурс]. – Режим доступа: <https://www.ixbt.com/cpu/x86-cpu-faq-2006.shtml>. – Дата доступа: 17.10.2023.
- [3] Computer History – The Birth of the Intel x86 Architecture [Электронный ресурс]. – Режим доступа: <https://unclespld3r.github.io/posts/2023/05/computer-history-the-birth-of-the-intel-x86-architecture/>. – Дата доступа: 28.10.2023.
- [4] RISC AND CISC [Электронный ресурс]. – Режим доступа: <https://arxiv.org/abs/1101.5364>. – Дата доступа: 28.10.2023.
- [5] AMD Ryzen 7 5800H [Электронный ресурс]. – Режим доступа: <https://technical.city/ru/cpu/Ryzen-7-5800H>. – Дата доступа: 28.10.2023.
- [6] Архитектура и система команд RISC-процессоров семейства ARM [Электронный ресурс]. – Режим доступа: <http://www.gaw.ru/html.cgi/txt/doc/micros/arm/index.htm>. – Дата доступа: 18.11.2023.
- [7] Arm documentation [Электронный ресурс]. – Режим доступа: <https://developer.arm.com/documentation/>. – Дата доступа: 30.11.2023.
- [8] Apple M1 [Электронный ресурс]. – Режим доступа: <https://www.apple.com/newsroom/2020/11/apple-unleashes-m1/>. – Дата доступа: 20.11.2023.
- [9] VirtualBox [Электронный ресурс]. – Режим доступа: <https://www.virtualbox.org>. – Дата доступа: 21.11.2023.
- [10] Oracle VM VirtualBox User Manual [Электронный ресурс]. – Режим доступа: <https://www.virtualbox.org/manual/UserManual.html>. – Дата доступа: 01.12.2023.
- [11] Tanenbaum, A. S. Structured Computer Organization, Fifth Edition / A. S. Tanenbaum. – Upper Saddle River, NJ. : Pearson Education, Inc., 2006. – 801 с.
- [12] RISC-архитектура процессора [Электронный ресурс]. – Режим доступа: <https://fb.ru/article/350271/risc-arhitektura-protsessora>. – Дата доступа: 22.11.2023.
- [13] SSE Instructions [Электронный ресурс]. – Режим доступа: https://docs.oracle.com/cd/E37838_01/html/E61064/eojde.html. – Дата доступа: 23.11.2023.

[14] SSE instruction set [Электронный ресурс]. – Режим доступа: <https://www.programmersought.com/article/39849958240/>. – Дата доступа: 01.12.2023.

[15] ARM NEON instruction set summary [Электронный ресурс]. – Режим доступа: <https://www.programmersought.com/article/16263303816/>.

[16] Arm NEON programming quick reference [Электронный ресурс]. – Режим доступа: <https://community.arm.com/arm-community-blogs/b/operating-systems-blog/posts/arm-neon-programming-quick-reference?pifragment-19720=1>. – Дата доступа: 01.12.2023.

[17] Алгоритм Левенштейна [Электронный ресурс]. – Режим доступа: https://ru.hexlet.io/courses/algorithms-graphs/lessons/levenshtein-distance/theory_unit. – Дата доступа: 01.12.2023.

[18] The Levenshtein Algorithm [Электронный ресурс]. – Режим доступа: <https://medium.com/@binaniharsj/the-levenshtein-algorithm-215567a3ab1f>. – Дата доступа: 01.12.2023.

[19] Fast Fourier transform [Электронный ресурс]. – Режим доступа: <https://cp-algorithms.com/algebra/fft.html>. – Дата доступа: 25.11.2023.

[20] Introduction to the Fast Fourier Transform [Электронный ресурс]. – Режим доступа: <https://cpp.helpful.codes/algorithms/FFT-Algorithm/>. – Дата доступа: 01.12.2023.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

Листинг А.1 – Стандартная реализация алгоритма Левенштейна

```
int levenshteinDistance(const std::string& str1, const std::string& str2) {
    const size_t len1 = str1.size();
    const size_t len2 = str2.size();
    // Создаем матрицу для хранения промежуточных результатов
    std::vector<std::vector<int>> dp(len1 + 1, std::vector<int>(len2 + 1,
0));
    // Инициализируем первую строку и первый столбец
    for (size_t i = 0; i <= len1; ++i) {
        dp[i][0] = static_cast<int>(i);
    }
    for (size_t j = 0; j <= len2; ++j) {
        dp[0][j] = static_cast<int>(j);
    }
    // Заполняем матрицу по правилам алгоритма
    for (size_t i = 1; i <= len1; ++i) {
        for (size_t j = 1; j <= len2; ++j) {
            int cost = (str1[i - 1] == str2[j - 1]) ? 0 : 1;
            dp[i][j] = std::min({
                dp[i - 1][j] + 1,      // удаление
                dp[i][j - 1] + 1,      // вставка
                dp[i - 1][j - 1] + cost // замена
            });
        }
    }
    return dp[len1][len2];
}
```

Листинг А.2 – Реализация алгоритма Левенштейна с SSE инструкциями

```
int levenshteinSSE(const std::string& str1, const std::string& str2) {
    const size_t len1 = str1.size();
    const size_t len2 = str2.size();
    __m128i* row1 = (__m128i*)malloc((len2 + 1) * sizeof(__m128i));
    __m128i* row2 = (__m128i*)malloc((len2 + 1) * sizeof(__m128i));
    for (size_t j = 0; j <= len2; ++j) {
        row1[j] = _mm_set_epi32(j, 0, 0, 0);
    }
    for (size_t i = 1; i <= len1; ++i) {
        row2[0] = _mm_set_epi32(i, 0, 0, 0);
        for (size_t j = 1; j <= len2; j += 4) {
            __m128i cost = _mm_cmpeq_epi32(_mm_loadu_si128((__m128i*) &
str1[i - 1]), _mm_loadu_si128((__m128i*) & str2[j - 1]));
            __m128i insert = _mm_add_epi32(_mm_load_si128(&row2[j - 1]),
_mm_set_epi32(1, 1, 1, 1));
            __m128i del = _mm_add_epi32(_mm_load_si128(&row1[j]),
_mm_set_epi32(1, 1, 1, 1));
            __m128i replace = _mm_add_epi32(_mm_load_si128(&row1[j - 1]),
_mm_sub_epi32(_mm_set_epi32(0, 0, 0, 0), cost));
            __m128i min1 = _mm_min_epi32(insert, del);
            __m128i min2 = _mm_min_epi32(min1, replace);
            _mm_store_si128(&row2[j], min2);
        }
        std::swap(row1, row2);
    }
}
```

```

    int result = _mm_cvtsi128_si32(row1[len2]);
    free(row1);
    free(row2);
    return result;
}

```

Листинг А.3 – Реализация алгоритма Левенштейна с *NEON* инструкциями

```

int levenshteinDistanceNEON(const std::string& str1, const std::string& str2)
{
    const size_t len1 = str1.size();
    const size_t len2 = str2.size();
    std::vector<std::vector<int>> dp(len1 + 1, std::vector<int>(len2 + 1,
0));
    for (size_t i = 0; i <= len1; ++i) {
        dp[i][0] = static_cast<int>(i);
    }
    for (size_t j = 0; j <= len2; ++j) {
        dp[0][j] = static_cast<int>(j);
    }
    for (size_t i = 1; i <= len1; ++i) {
        int prev_diag = i - 1;
        for (size_t j = 1; j <= len2; ++j) {
            int cost = (str1[i - 1] == str2[j - 1]) ? 0 : 1;
            int32x4_t insert = vld1q_s32(&dp[i][j - 1]);
            int32x4_t del = vld1q_s32(&dp[i - 1][j]);
            int32x4_t replace = vld1q_s32(&dp[prev_diag][j - 1]);
            insert = vaddq_s32(insert, vdupq_n_s32(1));
            del = vaddq_s32(del, vdupq_n_s32(1));
            uint32x4_t cmp = vceqq_s32(vld1q_s32((int*)&str1[i - 1]),
vld1q_s32((int*)&str2[j - 1]));
            replace = vaddq_s32(replace, vreinterpretq_s32_u32(cmp));
            int32x4_t min1 = vminq_s32(insert, del);
            int32x4_t min2 = vminq_s32(min1, replace);
            vst1q_s32(&dp[i][j], min2);
            prev_diag = vgetq_lane_s32(replace, 3);
        }
    }
    return dp[len1][len2];
}

```

Листинг А.4 – Стандартная реализация алгоритма БПФ

```

vector<complex<double>> FFT(vector<complex<double>>& x) {
    int N = x.size();
    vector<complex<double>> X(N);
    // Base case (N = 1)
    if (N == 1) {
        X[0] = x[0];
        return X;
    }
    // Split x into even and odd parts
    vector<complex<double>> x_even(N / 2), x_odd(N / 2);
    for (int i = 0; i < N / 2; ++i) {
        x_even[i] = x[2 * i];
        x_odd[i] = x[2 * i + 1];
    }
    // Compute the FFT of the even and odd parts
    vector<complex<double>> X_even = FFT(x_even);
    vector<complex<double>> X_odd = FFT(x_odd);
    // Combine the two FFTs to obtain the final FFT
    for (int k = 0; k < N / 2; ++k) {

```

```

        complex<double> w = exp(complex<double>(0, -2 * M_PI * k / N));
        X[k] = X_even[k] + w * X_odd[k];
        X[k + N / 2] = X_even[k] - w * X_odd[k];
    }
    return X;
}

```

Листинг A.5 – Реализация алгоритма БПФ с SSE инструкциями

```

void fftSSE(std::vector<std::complex<double>>& data) {
    int size = static_cast<int>(data.size());
    // Ensure the input size is a power of two
    if (size & (size - 1)) {
        data = ensurePowerOfTwo(data);
        size = static_cast<int>(data.size());
    }
    // Create temporary vectors to store real and imaginary parts
    std::vector<double> real(size);
    std::vector<double> imag(size);
    // Separate real and imaginary parts
    for (int i = 0; i < size; ++i) {
        real[i] = data[i].real();
        imag[i] = data[i].imag();
    }
    // Create complex vectors using SSE
    __m128d* xmm_real = (__m128d*) &real[0];
    __m128d* xmm_imag = (__m128d*) &imag[0];
    // Perform FFT using SSE
    for (int step = 1; step < size; step *= 2) {
        for (int i = 0; i < size; i += 2 * step) {
            for (int j = 0; j < step; ++j) {
                // Butterfly operation
                __m128d wr = _mm_setr_pd(cos(-2.0 * M_PI * j / (2 * step)),
cos(-2.0 * M_PI * (j + step) / (2 * step)));
                __m128d wi = _mm_setr_pd(sin(-2.0 * M_PI * j / (2 * step)),
sin(-2.0 * M_PI * (j + step) / (2 * step)));
                // Load values
                __m128d even_real = _mm_load_pd(&real[i + j]);
                __m128d even_imag = _mm_load_pd(&imag[i + j]);
                __m128d odd_real = _mm_load_pd(&real[i + j + step]);
                __m128d odd_imag = _mm_load_pd(&imag[i + j + step]);
                // Multiply by twiddle factor
                __m128d odd_real_twiddled = _mm_mul_pd(odd_real, wr);
                __m128d odd_imag_twiddled = _mm_mul_pd(odd_imag, wi);
                // Calculate FFT
                __m128d result_real = _mm_add_pd(even_real,
odd_real_twiddled);
                __m128d result_imag = _mm_add_pd(even_imag,
odd_imag_twiddled);

                // Store results
                _mm_store_pd(&real[i + j], result_real);
                _mm_store_pd(&imag[i + j], result_imag);
            }
        }
    }
    // Combine real and imaginary parts back into complex numbers
    for (int i = 0; i < size; ++i) {
        data[i] = std::complex<double>(real[i], imag[i]);
    }
}

```


Листинг А.6 – Реализация алгоритма БПФ с *NEON* инструкциями

```
void fftNEON(std::vector<std::complex<double>>& data) {
    int size = static_cast<int>(data.size());

    // Ensure the input size is a power of two
    if (size & (size - 1)) {
        data = ensurePowerOfTwo(data);
        size = static_cast<int>(data.size());
    }
    // Create temporary vectors to store real and imaginary parts
    std::vector<double> real(size);
    std::vector<double> imag(size);
    // Separate real and imaginary parts
    for (int i = 0; i < size; ++i) {
        real[i] = data[i].real();
        imag[i] = data[i].imag();
    }
    // Perform FFT using NEON intrinsics
    for (int step = 1; step < size; step *= 2) {
        for (int i = 0; i < size; i += 2 * step) {
            for (int j = 0; j < step; ++j) {
                // Butterfly operation
                float32x2_t wr = vcreate_f32(
                    static_cast<float>(cos(-2.0 * M_PI * j / (2 * step))),
                    static_cast<float>(cos(-2.0 * M_PI * (j + step) / (2 *
step))));
                float32x2_t wi = vcreate_f32(
                    static_cast<float>(sin(-2.0 * M_PI * j / (2 * step))),
                    static_cast<float>(sin(-2.0 * M_PI * (j + step) / (2 *
step))));
                // Load values
                float32x2_t even_real = vld1_f32(&real[i + j]);
                float32x2_t even_imag = vld1_f32(&imag[i + j]);
                float32x2_t odd_real = vld1_f32(&real[i + j + step]);
                float32x2_t odd_imag = vld1_f32(&imag[i + j + step]);
                // Multiply by twiddle factor
                float32x2_t odd_real_twiddled = vmul_f32(odd_real, wr);
                float32x2_t odd_imag_twiddled = vmul_f32(odd_imag, wi);
                // Calculate FFT
                float32x2_t result_real = vadd_f32(even_real,
odd_real_twiddled);
                float32x2_t result_imag = vadd_f32(even_imag,
odd_imag_twiddled);
                // Store results
                vst1_f32(&real[i + j], result_real);
                vst1_f32(&imag[i + j], result_imag);
            }
        }
    }
    // Combine real and imaginary parts back into complex numbers
    for (int i = 0; i < size; ++i) {
        data[i] = std::complex<double>(real[i], imag[i]);
    }
}
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Функциональная схема алгоритма

ПРИЛОЖЕНИЕ В
(обязательное)
Блок-схема алгоритма

ПРИЛОЖЕНИЕ Г
(обязательное)
Скриншоты сравнений производительности

ПРИЛОЖЕНИЕ Д
(обязательное)
Ведомость документов